



Trabajo Final Integrador - Foodstore

Institucion: UTN

Carrera: Tecnicatura Universitaria en Programacion

Materia: Programación 3

Alumno: Agustin Chiaravalloti

Comision: 2

Índice

Índice.....	2
1. Introducción.....	3
2. Marco Teórico.....	4
Tecnologías de Backend.....	4
Tecnologías de Frontend.....	5
Conceptos Clave Aplicados.....	5
3. Arquitectura y Decisiones Técnicas.....	6
Arquitectura en capas Backend.....	6
Patron DTO.....	6
Entidad y Repositorio Base.....	7
Manejo Global de Excepciones.....	7
Gestión Transaccional de Pedidos.....	8
Comunicación Frontend-Backend.....	8
4. Modelo de Datos.....	9
Entidades	10
Relaciones.....	11
Enumeraciones.....	11
5. Funcionalidades del Sistema.....	12
Módulo de Autenticación.....	12
Módulo de tienda.....	14
Módulo de Carrito y Confirmación de Compra.....	15
Módulo de Seguimiento de Pedidos (Cliente).....	17
Módulo de Administración.....	17
6. Dificultades y Soluciones.....	20
Atomicidad en la creacion de pedidos.....	20
Configuración del entorno Frontend.....	20
Comunicación entre Frontend y Backend.....	20
7. Conclusion.....	21
8. Bibliografía/Webgrafía.....	22

1. Introducción.

El presente trabajo documenta el desarrollo de Food Store, un sistema de gestión de pedidos de comida implementado como una aplicación web full stack. El proyecto fue desarrollado en el marco de la materia Programación 3 de la Tecnicatura Universitaria en Programación a Distancia, con el objetivo de integrar y aplicar los conocimientos adquiridos sobre desarrollo backend, frontend y comunicación entre ambos mediante una API REST.

El sistema permite la gestión integral de un negocio de comidas a través de dos perfiles de usuario diferenciados. Por un lado, el perfil Administrador cuenta con un panel de administración que habilita operaciones CRUD sobre categorías y productos, así como la gestión del estado de los pedidos. Por otro lado, el perfil Usuario permite registrarse, iniciar sesión, navegar el catálogo de productos, administrar un carrito de compras persistente, confirmar pedidos y consultar el historial y estado de los mismos.

El desarrollo se abordó siguiendo una metodología ágil basada en Scrum, organizando las funcionalidades en épicas e historias de usuario con sus respectivos criterios de aceptación. Cada historia fue traducida en pantallas concretas del frontend y en endpoints de la API REST del backend, respetando las validaciones y reglas de negocio definidas. El resultado es un producto navegable y verificable que refleja los flujos principales del sistema: el flujo de compra del cliente y el flujo de gestión del administrador.

2. Marco Teórico.

El desarrollo de Food Store involucró un conjunto de tecnologías tanto del lado del servidor (backend) como del lado del cliente (frontend). A continuación se describen las principales herramientas utilizadas y los conceptos clave aplicados.

Tecnologías de Backend.

Java es el lenguaje de programación utilizado para el backend. Se trata de un lenguaje orientado a objetos, fuertemente tipado y multiplataforma, ampliamente adoptado en el desarrollo de aplicaciones empresariales. Para este proyecto se utilizó Java 21.

Spring Boot es un framework basado en Java que facilita la creación de aplicaciones autónomas y de nivel productivo con una configuración mínima. Proporciona un servidor web embebido y un sistema de inyección de dependencias que permite estructurar el código de forma desacoplada y mantenible. Fue la base sobre la cual se construyó la API REST del sistema.

Spring Data JPA es el módulo de Spring que simplifica el acceso a datos. Mediante el uso de repositorios, permite realizar operaciones de persistencia (crear, leer, actualizar y eliminar) sin necesidad de escribir consultas SQL manualmente, generando las consultas automáticamente a partir de los nombres de los métodos.

Hibernate es la implementación de JPA utilizada por defecto en Spring. Actúa como un ORM (Object-Relational Mapping), traduciendo las clases Java (entidades) en tablas de la base de datos y los objetos en filas, gestionando automáticamente la creación del esquema y las relaciones entre tablas.

H2 Database es una base de datos relacional en memoria utilizada durante el desarrollo. Su principal ventaja es que no requiere instalación y se inicializa automáticamente al arrancar la aplicación, lo que agiliza las pruebas. El esquema se genera de forma automática a partir de las entidades.

Gradle es la herramienta de automatización de compilación y gestión de dependencias empleada en el backend, encargada de descargar las librerías necesarias y ejecutar la aplicación.

Tecnologías de Frontend.

TypeScript es un superconjunto de JavaScript que incorpora tipado estático. Permite detectar errores en tiempo de desarrollo y escribir código más robusto y mantenible. Fue el lenguaje utilizado para implementar la lógica del lado del cliente.

Vite es una herramienta de construcción y servidor de desarrollo para aplicaciones web modernas. Ofrece recarga instantánea de los cambios (Hot Module Replacement), lo que acelera notablemente el ciclo de desarrollo del frontend.

Tailwind CSS es un framework de CSS basado en clases utilitarias. En lugar de escribir hojas de estilo separadas, permite aplicar estilos directamente en el marcado HTML mediante clases predefinidas, agilizando el diseño de interfaces consistentes.

Conceptos Clave Aplicados.

API REST es un estilo de arquitectura para el diseño de servicios web que utiliza el protocolo HTTP y sus verbos (GET, POST, PUT, DELETE) para realizar operaciones sobre recursos. La comunicación entre el frontend y el backend se realiza íntegramente a través de una API REST que intercambia datos en formato JSON.

DTO (Data Transfer Object) es un patrón de diseño que consiste en objetos cuyo único propósito es transportar datos entre las capas de la aplicación. Se utilizaron para definir con precisión qué información ingresa y egresa de la API, evitando exponer directamente las entidades del modelo y protegiendo datos sensibles como las contraseñas.

BCrypt es un algoritmo de hashing diseñado específicamente para el almacenamiento seguro de contraseñas. Aplica una función irreversible junto con un valor aleatorio (salt), de modo que las contraseñas nunca se almacenan en texto plano en la base de datos.

Soft Delete (eliminación lógica) es una técnica mediante la cual los registros no se eliminan físicamente de la base de datos, sino que se marcan como eliminados a través de un campo booleano. Esto permite preservar la integridad histórica de los datos.

3. Arquitectura y Decisiones Técnicas

La solución fue diseñada siguiendo principios de separación de responsabilidades y reutilización de código, tanto en el backend como en el frontend. A continuación se detallan las principales decisiones de arquitectura adoptadas y su justificación.

Arquitectura en capas Backend.

El backend se estructuró siguiendo una **arquitectura en capas**, donde cada capa tiene una responsabilidad única y bien definida. Esta organización facilita el mantenimiento, las pruebas y la posibilidad de modificar una capa sin afectar a las demás.

- **Capa de Modelo (model):** contiene las entidades JPA que representan las tablas de la base de datos (Usuario, Categoría, Producto, Pedido, DetallePedido).
- **Capa de Repositorio (repository):** se encarga del acceso a datos mediante Spring Data JPA. Define las interfaces que comunican la aplicación con la base de datos.
- **Capa de Servicio (service):** contiene la lógica de negocio. Aquí se aplican las validaciones, los cálculos y las reglas del sistema.
- **Capa de Controlador (controller):** expone los endpoints de la API REST, recibe las peticiones HTTP y delega el procesamiento a la capa de servicio.

Una petición típica atraviesa estas capas en orden (controlador → servicio → repositorio → base de datos) y la respuesta regresa por el mismo camino, garantizando un flujo de datos ordenado y predecible.

Patron DTO.

Se adoptó el patrón **DTO (Data Transfer Object)** para desacoplar la representación interna de los datos (las entidades) de la información que se intercambia con el cliente. Para cada entidad se definieron DTOs específicos según el contexto: uno

para la creación (datos de entrada con sus validaciones), otro para la actualización (con campos opcionales para permitir ediciones parciales) y otro para la respuesta (datos de salida).

Esta decisión aporta dos beneficios concretos. En primer lugar, permite aplicar validaciones precisas sobre los datos que ingresan. En segundo lugar, evita exponer información sensible: el DTO de respuesta del usuario, por ejemplo, no incluye el campo de contraseña, garantizando que esta nunca viaje en las respuestas de la API.

Entidad y Repositorio Base.

Con el objetivo de evitar la duplicación de código, se implementó una clase abstracta **Base** de la cual heredan todas las entidades del sistema. Esta clase centraliza los campos comunes: el identificador autogenerado, el campo de eliminación lógica, las marcas de tiempo de auditoría (fecha de creación y actualización) y el campo de versión para el control de concurrencia.

De manera análoga, se creó una interfaz **BaseRepository** que extiende la funcionalidad estándar de Spring Data JPA y centraliza la lógica del soft delete y la búsqueda de entidades. De este modo, los repositorios de cada entidad heredan automáticamente estas operaciones sin necesidad de reimplementarlas.

Manejo Global de Excepciones.

Para proporcionar respuestas de error consistentes en toda la API, se implementó un **manejador global de excepciones**. En lugar de gestionar los errores individualmente en cada controlador, esta clase centralizada intercepta las excepciones lanzadas en cualquier parte de la aplicación y las traduce a respuestas HTTP con el código de estado y el mensaje correspondientes (por ejemplo, 404 cuando un recurso no existe, o 400 cuando se viola una regla de negocio).

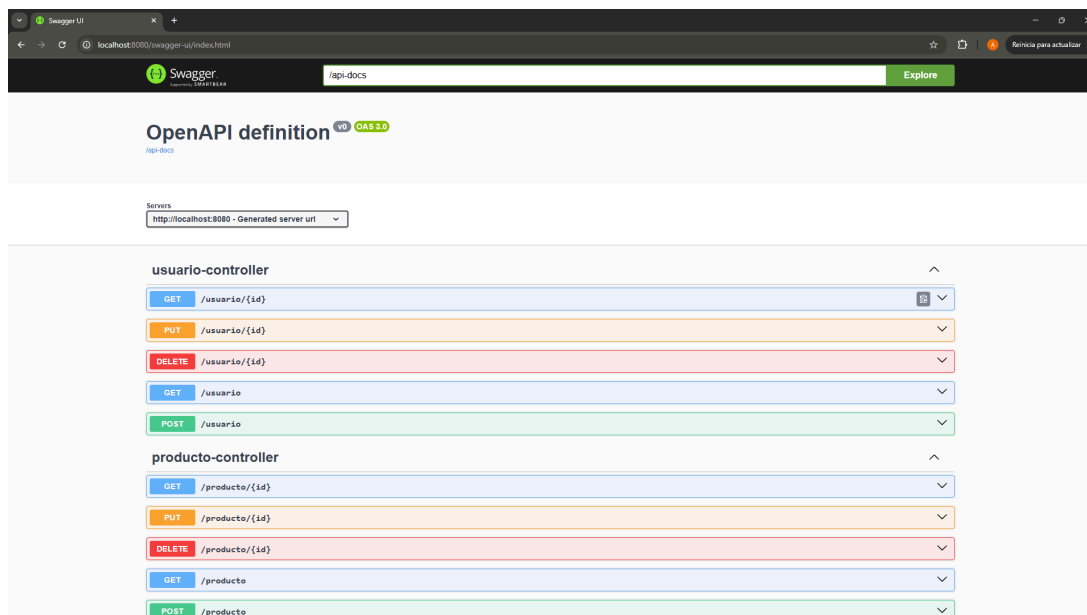
Gestión Transaccional de Pedidos.

La creación de un pedido es la operación más compleja del sistema, ya que implica múltiples acciones que deben ejecutarse de forma atómica: validar la disponibilidad y el stock de cada producto, calcular los subtotales y el total, y reducir el stock correspondiente. Para garantizar la integridad de los datos, esta operación se marcó como **transaccional**. De esta forma, si alguna validación falla a mitad del proceso (por ejemplo, si un producto no tiene stock suficiente), se revierten automáticamente todos los cambios realizados, evitando que el sistema quede en un

Comunicación Frontend-Backend.

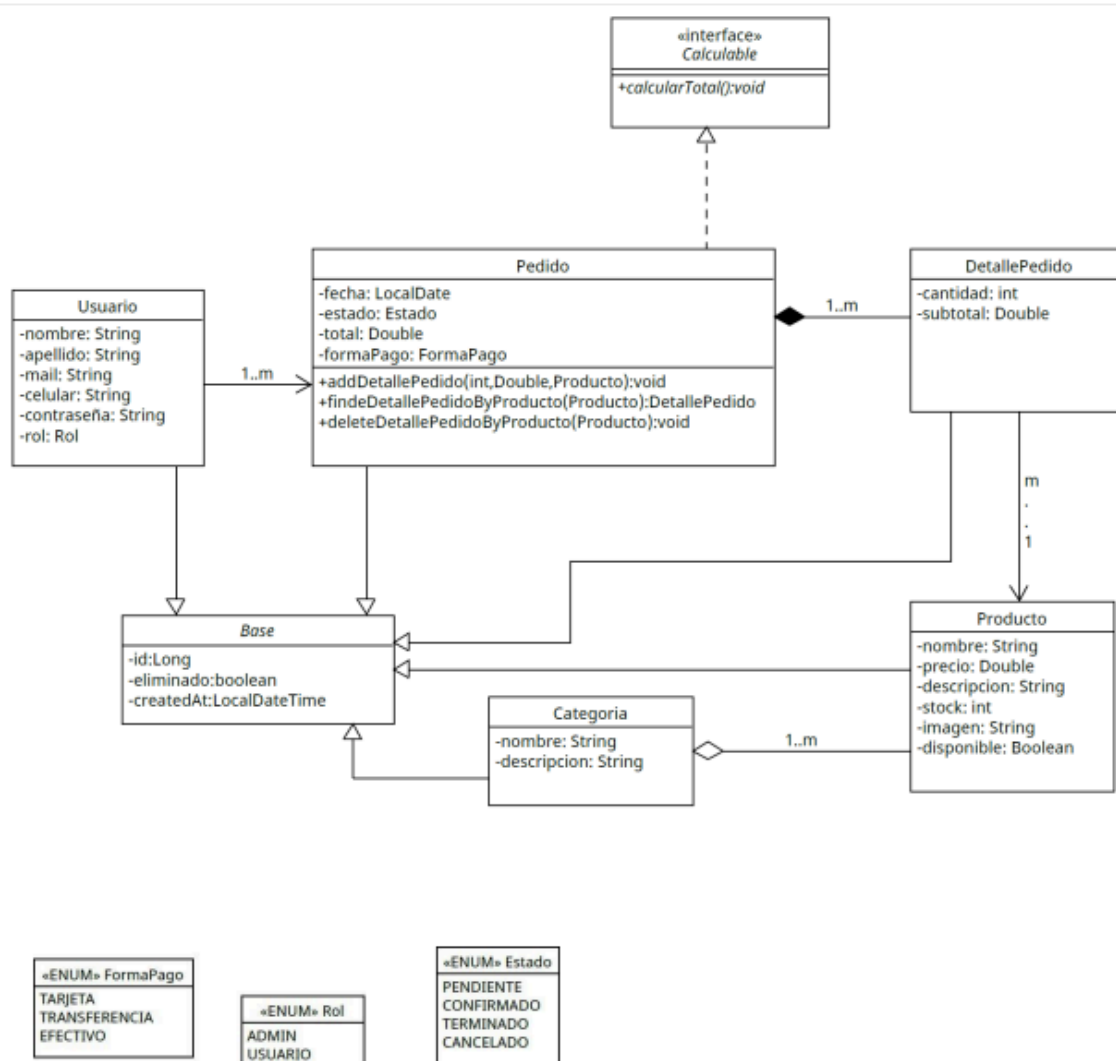
El frontend se comunica con el backend exclusivamente a través de la API REST, realizando peticiones HTTP mediante la función fetch del navegador. Para centralizar esta lógica, se desarrolló un módulo de API reutilizable que gestiona las peticiones y el manejo de errores de forma uniforme. La autenticación se resolvió de manera básica almacenando los datos del usuario en el localStorage del navegador, conforme a los lineamientos del trabajo, que indican que el sistema tiene fines educativos y no implementa seguridad de nivel productivo.

A continuación se presenta la documentación interactiva de la API generada automáticamente mediante SpringDoc OpenAPI:

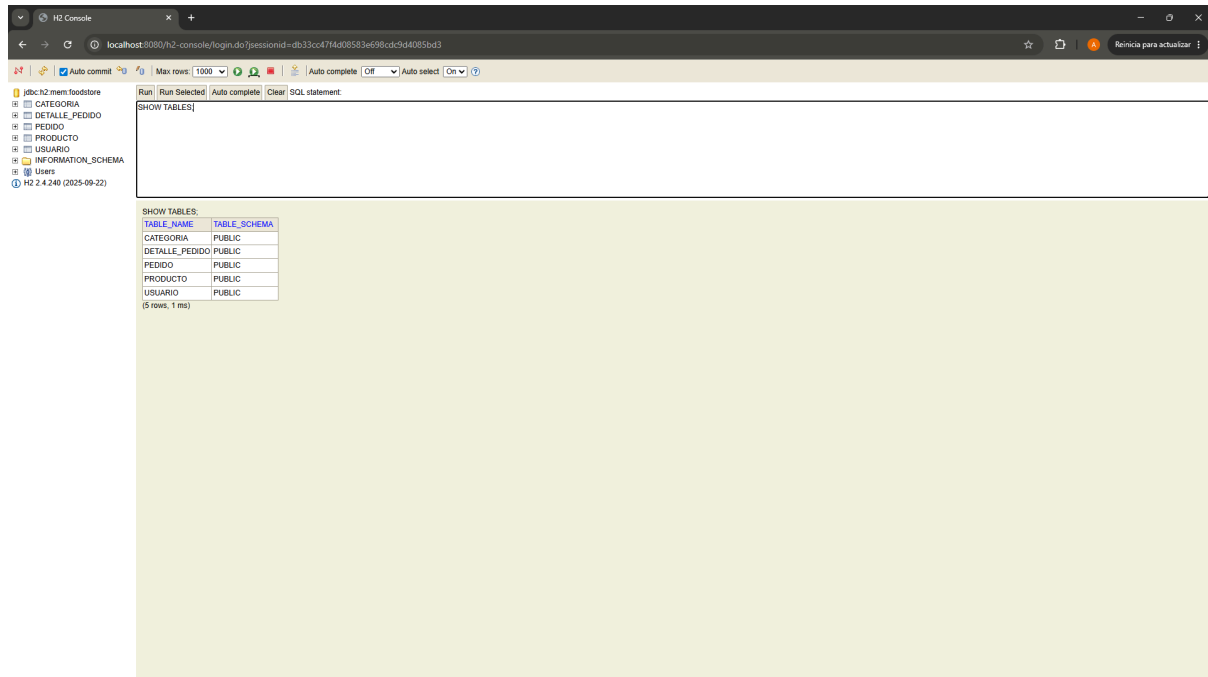


4. Modelo de Datos

El modelo de datos del sistema está compuesto por cinco entidades principales que heredan de una clase base común. A continuación se describen las entidades y las relaciones que existen entre ellas.



La siguiente captura muestra la estructura de tablas generada automáticamente por Hibernate a partir del modelo de clases:



Entidades .

- **Base:** clase abstracta de la que heredan todas las entidades. Aporta los campos comunes: identificador, estado de eliminación lógica, fechas de auditoría y versión.
- **Usuario:** representa a las personas que interactúan con el sistema. Contiene nombre, apellido, email, celular, contraseña (almacenada de forma encriptada) y un rol que determina sus permisos (ADMIN o USUARIO).
- **Categoría:** agrupa los productos para facilitar su organización y búsqueda. Contiene nombre y descripción.
- **Producto:** representa los artículos del catálogo. Contiene nombre, precio, descripción, stock, imagen y disponibilidad. Cada producto pertenece a una categoría.

- **Pedido:** representa una compra realizada por un usuario. Contiene la fecha, el estado, el total, la forma de pago y el conjunto de detalles que lo componen.
- **DetallePedido:** representa cada línea de un pedido, es decir, la relación entre un pedido y un producto específico con su cantidad y subtotal.

Relaciones.

El modelo presenta las siguientes relaciones:

- **Producto – Categoría (muchos a uno):** muchos productos pueden pertenecer a una misma categoría.
- **Pedido – Usuario (muchos a uno):** un usuario puede realizar muchos pedidos.
- **Pedido – DetallePedido (uno a muchos):** un pedido se compone de uno o varios detalles.
- **DetallePedido – Producto (muchos a uno):** cada detalle hace referencia a un producto.

Adicionalmente, la entidad Pedido implementa la interfaz **Calculable**, que define el comportamiento de cálculo del total a partir de la suma de los subtotales de sus detalles, encapsulando esta responsabilidad dentro de la propia entidad.

Enumeraciones.

El sistema utiliza tres enumeraciones para representar conjuntos de valores fijos:

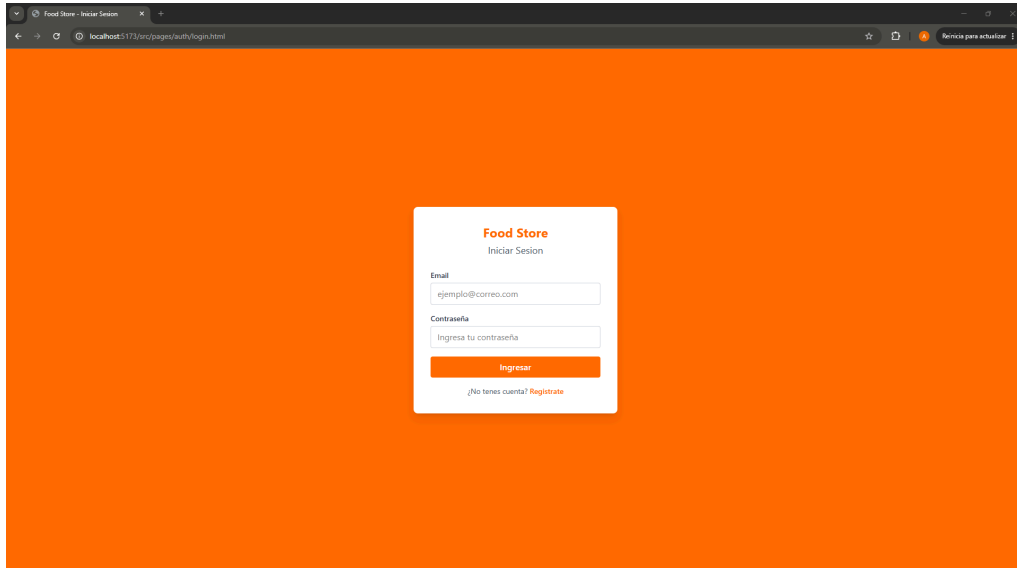
- **Rol:** ADMIN, USUARIO.
- **Estado** (de un pedido): PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO.
- **FormaPago:** TARJETA, TRANSFERENCIA, EFECTIVO.

5. Funcionalidades del Sistema

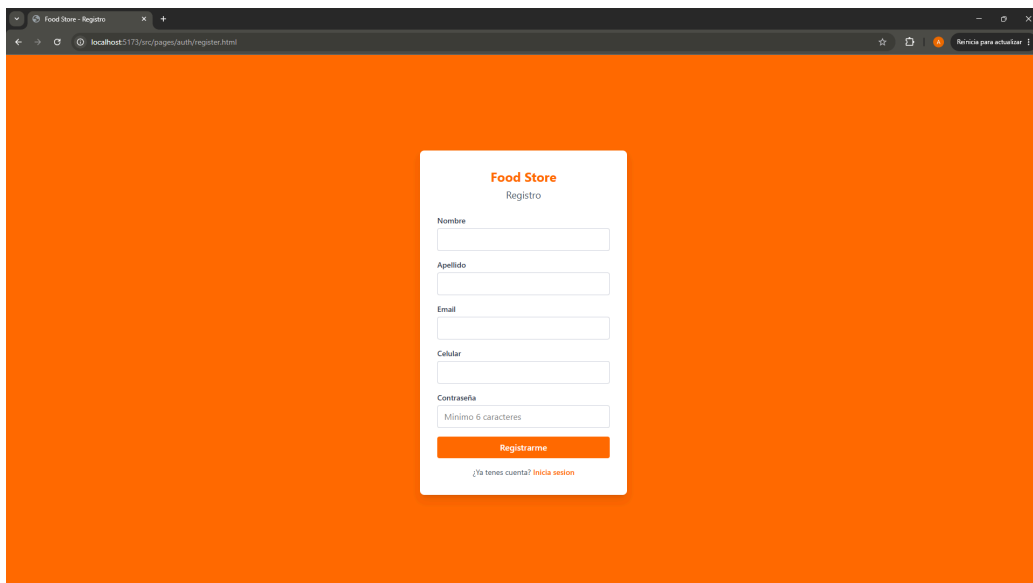
El sistema implementa la totalidad de las funcionalidades definidas en el backlog del proyecto, organizadas en cinco épicas y veintinueve historias de usuario. Estas funcionalidades se materializan en dos perfiles de acceso con responsabilidades diferenciadas —cliente (USUARIO) y administrador (ADMIN)— cuyos permisos son validados tanto en la interfaz como en la lógica de redirección de la aplicación. A continuación se describen los módulos que componen el sistema, detallando su comportamiento funcional y los endpoints de la API REST que los sustentan.

Módulo de Autenticación.

Gestiona el registro de nuevos usuarios y la validación de credenciales. El registro (`POST /usuario`) crea la cuenta asignando por defecto el rol USUARIO y persistiendo la contraseña mediante un hash BCrypt, garantizando que esta nunca se almacene ni se transmita en texto plano. El inicio de sesión (`POST /auth/login`) verifica las credenciales comparando la contraseña ingresada contra el hash almacenado y, ante un resultado satisfactorio, retorna los datos del usuario —sin incluir la contraseña— que se persisten en el navegador para el control de sesión. La aplicación implementa una redirección condicionada por rol, derivando al administrador al panel de gestión y al cliente al catálogo.



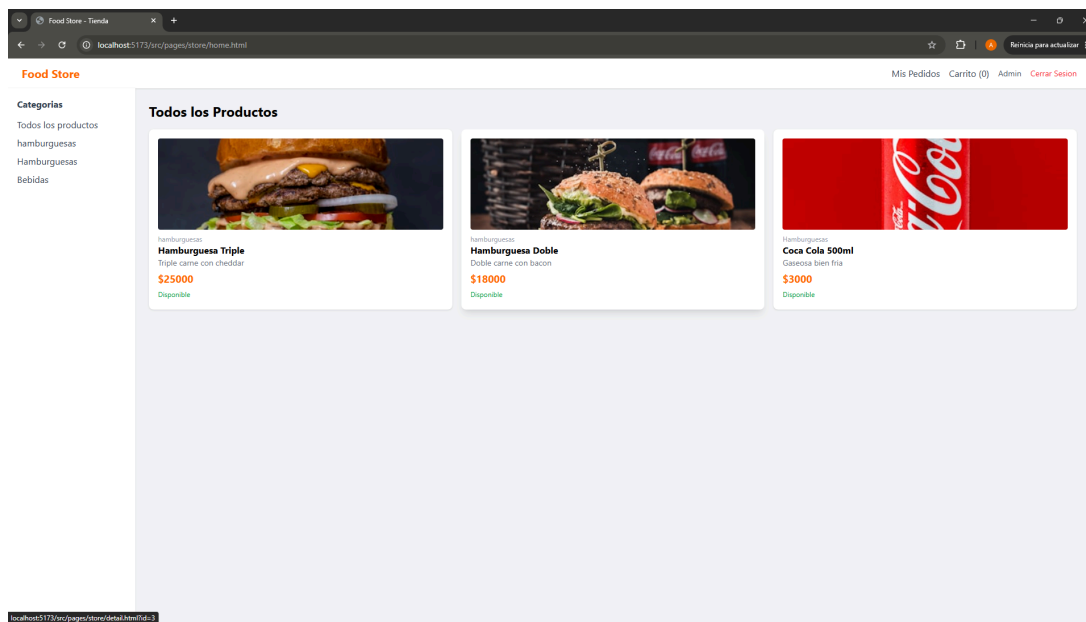
The screenshot shows a web browser window with the title "Food Store - Iniciar Sesión". The address bar shows "localhost:5173/src/pages/auth/login.html". The page has a solid orange background. In the center, there is a white card with the "Food Store" logo and the text "Iniciar Sesión". Below this, there are two input fields: "Email" with the placeholder "ejemplo@correo.com" and "Contraseña" with the placeholder "Ingresa tu contraseña". An orange button labeled "Ingresar" is positioned below the password field. At the bottom of the card, there is a link that says "¿No tienes cuenta? Regístrate".

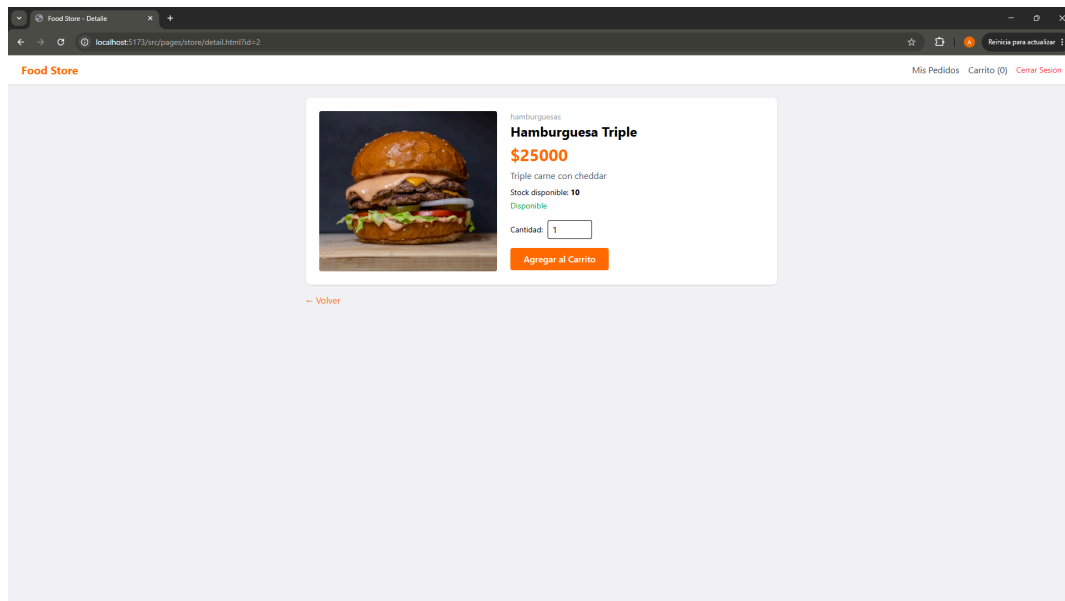


The screenshot shows a web browser window with the title "Food Store - Registro". The address bar shows "localhost:5173/src/pages/auth/register.html". The page has a solid orange background. In the center, there is a white card with the "Food Store" logo and the text "Registro". Below this, there are five input fields: "Nombre", "Apellido", "Email", "Celular", and "Contraseña". The "Contraseña" field has a placeholder "Mínimo 6 caracteres". An orange button labeled "Regístrate" is positioned below the password field. At the bottom of the card, there is a link that says "¿Ya tienes cuenta? Inicia sesión".

Módulo de tienda.

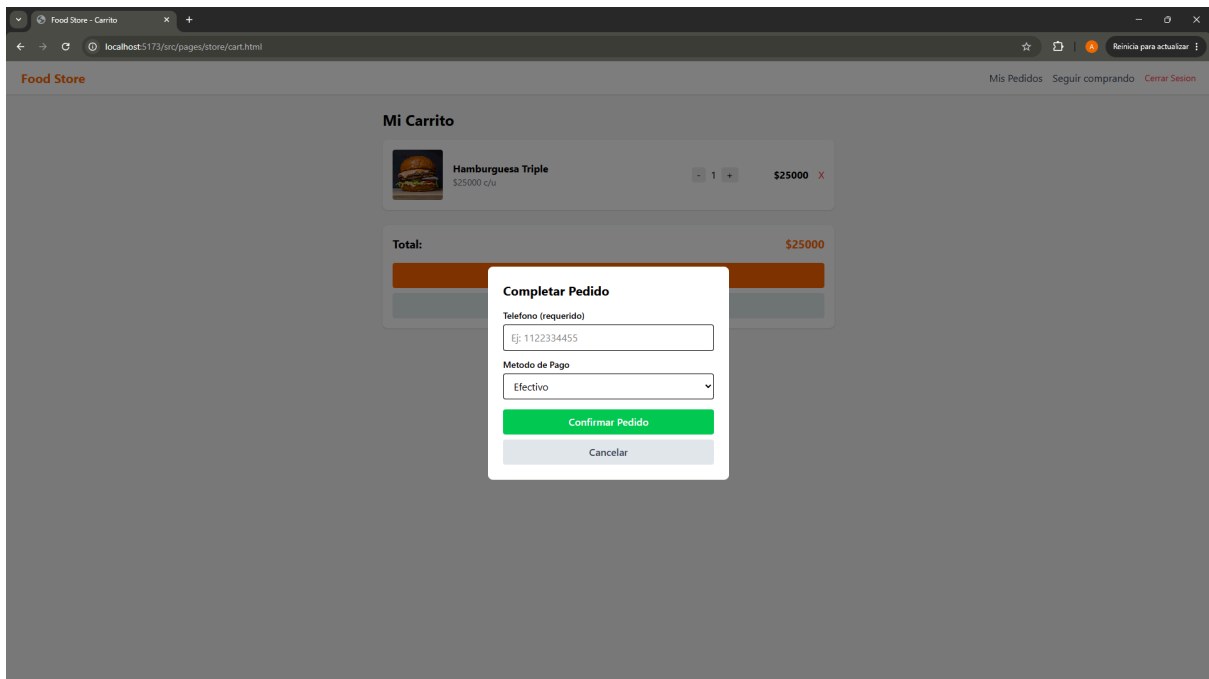
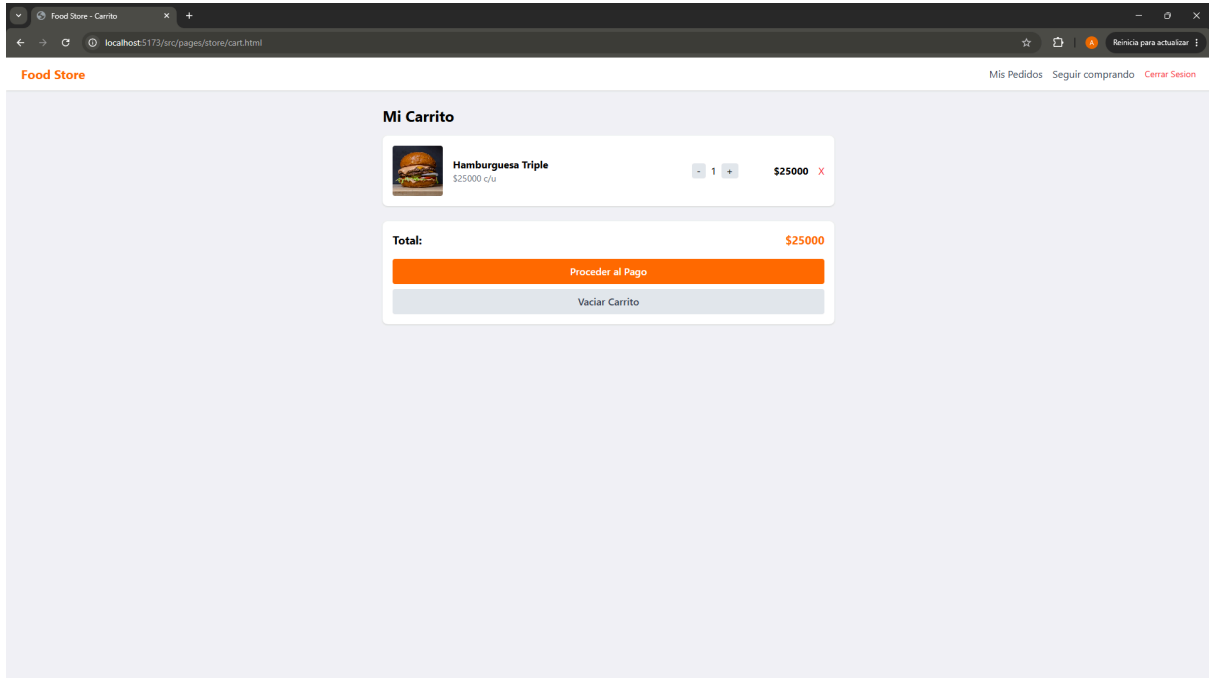
Constituye la vista principal del cliente. El catálogo (`GET /producto`) presenta los productos en una grilla que expone imagen, nombre, descripción, precio y estado de disponibilidad. Incorpora un filtrado por categoría resuelto mediante un endpoint dedicado (`GET /producto/categoria/{id}`), que consulta la relación entre producto y categoría a nivel de persistencia. La vista de detalle (`GET /producto/{id}`) presenta la información completa del artículo e incorpora un selector de cantidad con validación contra el stock disponible, impidiendo la incorporación al carrito de productos sin existencias o marcados como no disponibles.





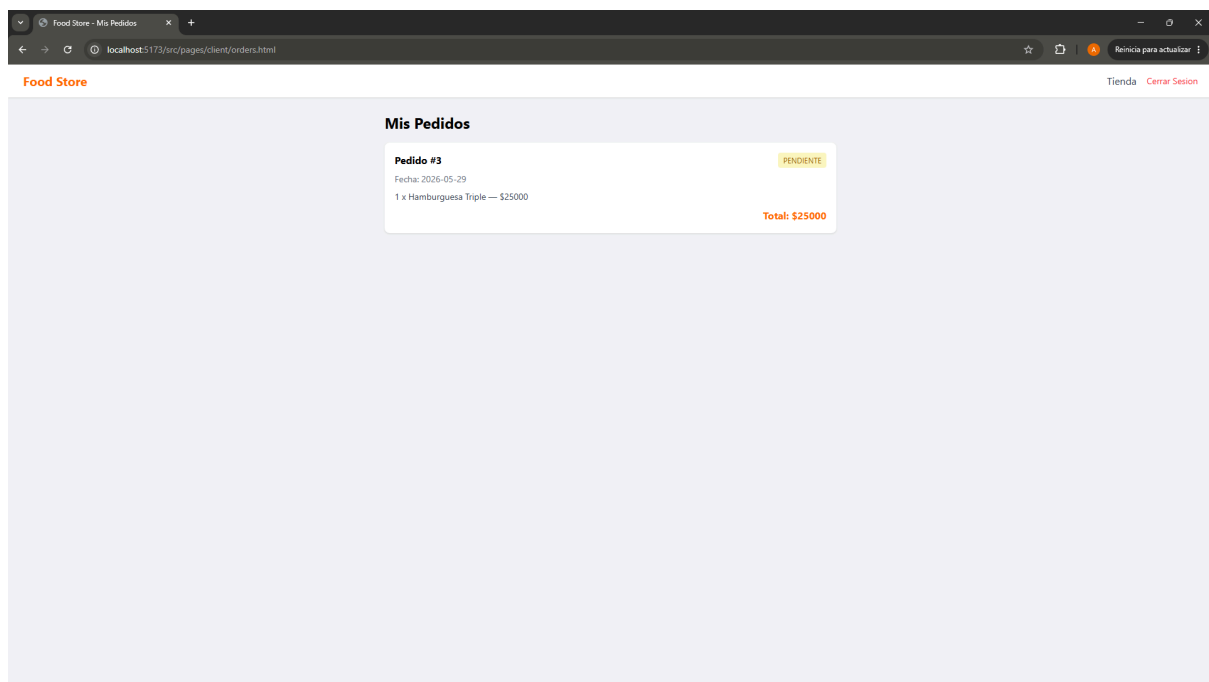
Módulo de Carrito y Confirmación de Compra.

El carrito de compras permite la revisión de los productos seleccionados, la modificación de cantidades y la eliminación de ítems, persistiendo su estado en el almacenamiento local del navegador para mantenerse disponible entre sesiones. La confirmación de la compra (**POST /pedido**) constituye la operación más relevante del flujo del cliente: el sistema valida la existencia, disponibilidad y stock de cada producto, calcula los subtotales y el total de forma automática, y reduce el stock de los artículos adquiridos. Toda la operación se ejecuta dentro de una transacción, lo que garantiza la atomicidad del proceso y la reversión completa de los cambios ante cualquier inconsistencia.



Módulo de Seguimiento de Pedidos (Cliente).

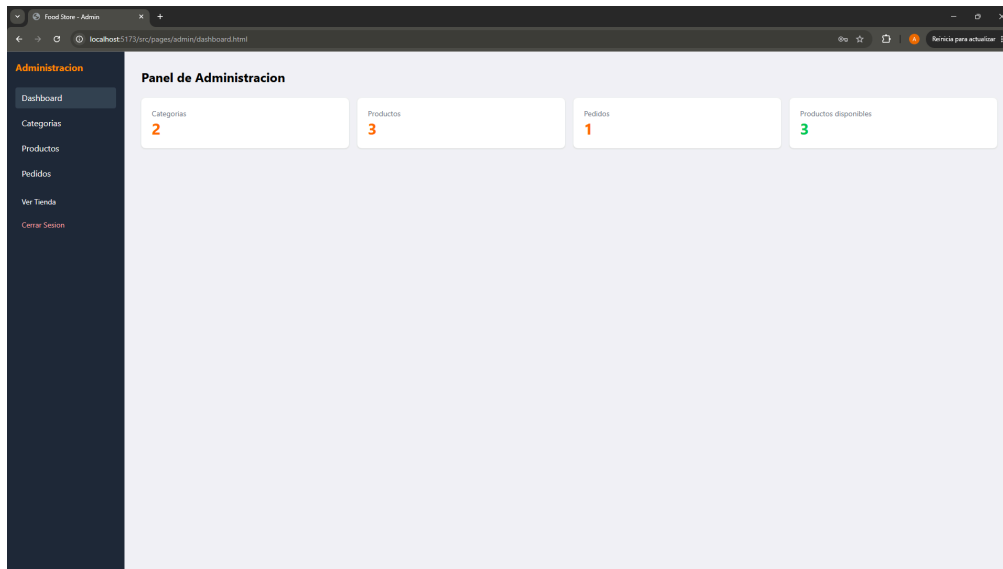
Habilita la consulta del historial de pedidos del usuario autenticado (`GET /pedido/usuario/{id}`), restringiendo el acceso exclusivamente a los pedidos propios. Cada pedido se presenta con su identificador, fecha, estado —diferenciado visualmente mediante un indicador de color—, el desglose de productos y el total, permitiendo al cliente realizar el seguimiento del ciclo de vida de sus compras.



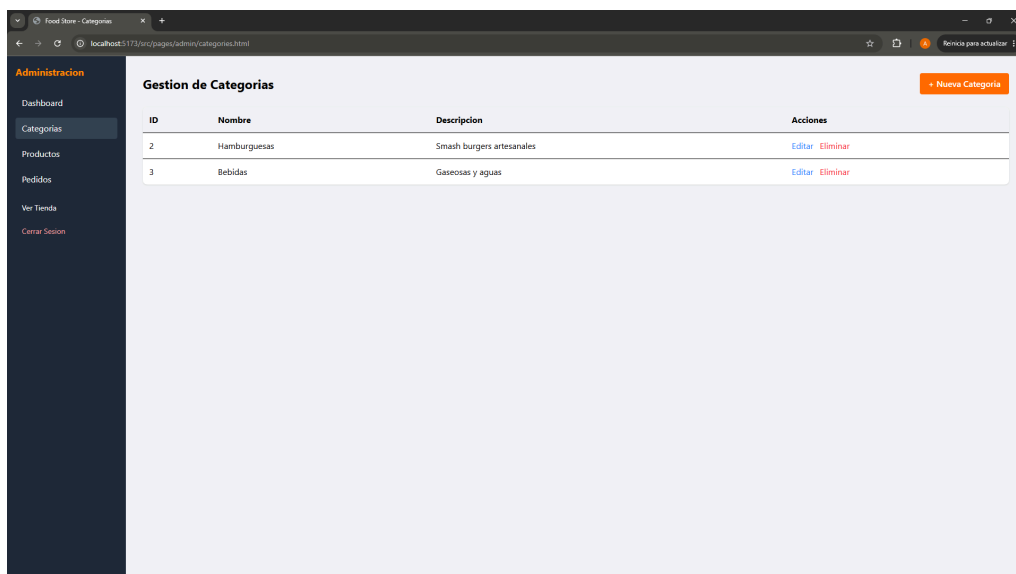
Módulo de Administración.

El panel de administración, de acceso restringido a usuarios con rol ADMIN, centraliza la gestión integral del sistema. La protección de la vista se implementa verificando el rol del usuario en sesión y redirigiendo a quienes no posean los permisos correspondientes. El módulo se compone de:

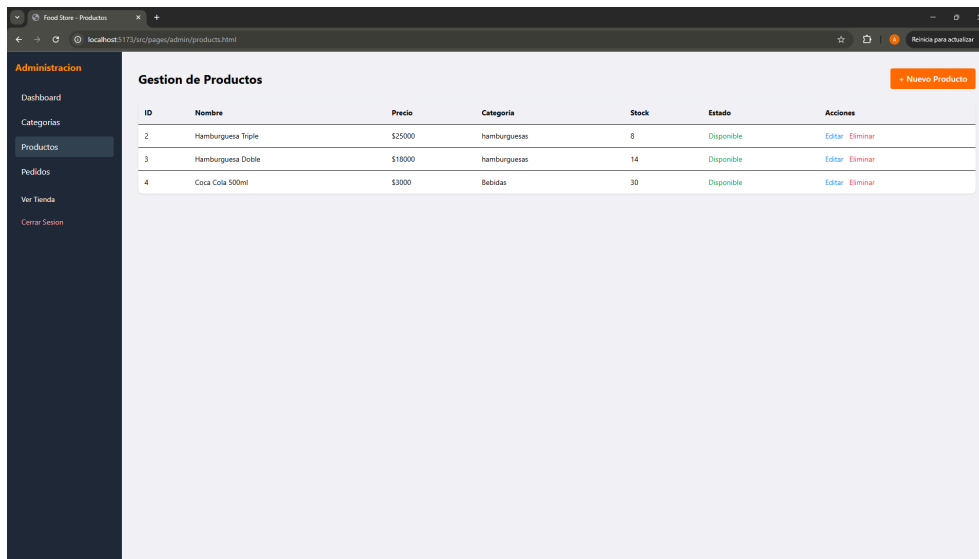
- **Dashboard:** presenta indicadores agregados del sistema (total de categorías, productos, pedidos y productos disponibles), obtenidos mediante consultas concurrentes a la API.



- **Gestión de Categorías:** implementa el ciclo CRUD completo (**POST**, **PUT**, **DELETE** /*categoria*) a través de una tabla y formularios modales, con eliminación lógica de los registros.

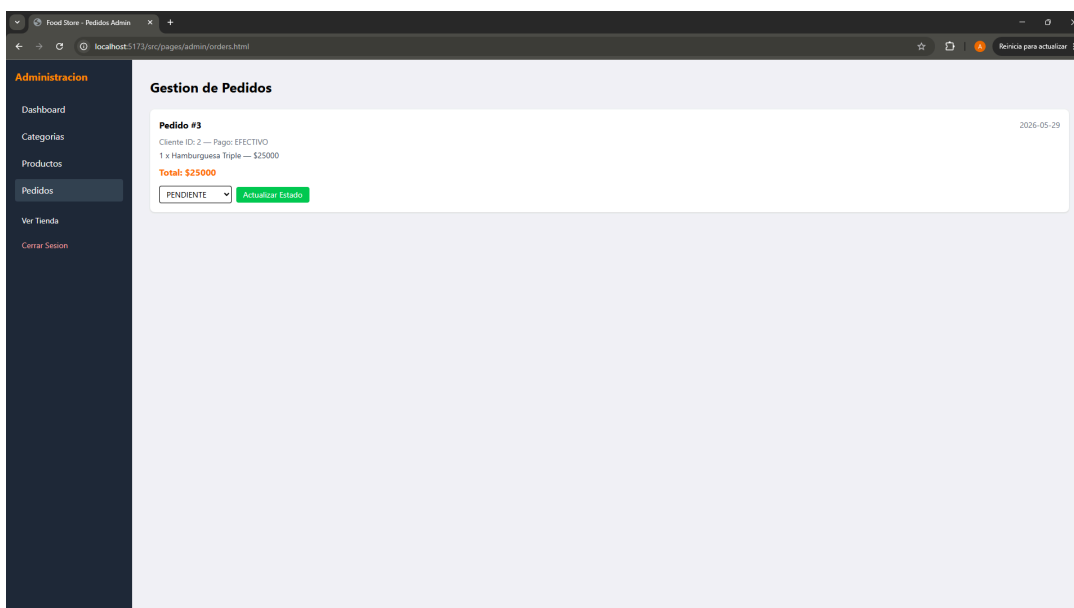


- **Gestión de Productos:** habilita el CRUD de productos (**POST**, **PUT**, **DELETE** **/producto**), incluyendo la asociación a una categoría existente, la gestión de stock y precio, y el control de disponibilidad.



ID	Nombre	Precio	Categoría	Stock	Estado	Acciones
2	Hamburguesa Triple	\$25000	hamburguesas	8	Disponible	Editar Eliminar
3	Hamburguesa Doble	\$18000	hamburguesas	14	Disponible	Editar Eliminar
4	Coca Cola 500ml	\$3000	Bebidas	30	Disponible	Editar Eliminar

- **Gestión de Pedidos:** expone la totalidad de los pedidos del sistema (**GET** **/pedido**) y permite la actualización de su estado (**PUT** **/pedido/{id}**), cambio que se propaga a la vista de seguimiento del cliente.



Gestion de Pedidos	
Pedido #3	2026-05-29
Cliente ID: 2 — Pago: EFECTIVO	
1 x Hamburguesa Triple — \$25000	
Total: \$25000	
PENDIENTE	Actualizar Estado

6. Dificultades y Soluciones

Durante el desarrollo del proyecto se presentaron diversos obstáculos técnicos, tanto conceptuales como de configuración del entorno. A continuación se documentan los más relevantes y la forma en que fueron resueltos.

Atomicidad en la creación de pedidos.

La creación de un pedido implica múltiples operaciones encadenadas (validación de stock, cálculo de totales y reducción de inventario). El riesgo era que, si una de estas operaciones fallaba a mitad del proceso, el sistema quedara en un estado inconsistente —por ejemplo, con el stock de algunos productos ya reducido pero sin un pedido válido creado. La solución fue marcar la operación como transaccional, delegando en el framework la reversión automática de todos los cambios ante cualquier excepción, garantizando así la integridad de los datos.

Configuración del entorno Frontend.

La creación de un pedido implica múltiples operaciones encadenadas (validación de stock, cálculo de totales y reducción de inventario). El riesgo era que, si una de estas operaciones fallaba a mitad del proceso, el sistema quedara en un estado inconsistente —por ejemplo, con el stock de algunos productos ya reducido pero sin un pedido válido creado. La solución fue marcar la operación como transaccional, delegando en el framework la reversión automática de todos los cambios ante cualquier excepción, garantizando así la integridad de los datos.

Comunicación entre Frontend y Backend.

Al conectar ambos proyectos surgieron errores de comunicación derivados de dos factores: por un lado, la política de mismo origen (CORS), que el navegador aplica al realizar peticiones entre distintos puertos; y por otro, la necesidad de mantener el servidor backend en ejecución durante el desarrollo del frontend. El primer punto se resolvió habilitando las peticiones de origen cruzado en los controladores, y el segundo se incorporó como parte del flujo de trabajo de desarrollo.

7. Conclusion

El desarrollo de Food Store permitió integrar de manera práctica los contenidos abordados a lo largo de la materia, abarcando el ciclo completo de construcción de una aplicación web full stack. Se logró implementar un backend robusto basado en una arquitectura en capas, una API REST funcional y un frontend que consume dichos servicios, cumpliendo con la totalidad de las épicas e historias de usuario definidas en el backlog del proyecto.

A nivel técnico, el proyecto consolidó la comprensión de conceptos fundamentales del desarrollo de software, tales como la separación de responsabilidades, el uso de patrones de diseño como DTO, la persistencia mediante un ORM, el manejo transaccional de operaciones críticas y la centralización del tratamiento de errores. Asimismo, la implementación del frontend permitió aplicar la comunicación asíncrona con una API y la gestión del estado de la aplicación del lado del cliente.

Más allá del resultado funcional, el principal aprendizaje radicó en comprender el porqué de cada decisión de diseño y no únicamente su implementación. La resolución de los obstáculos surgidos durante el desarrollo reforzó la capacidad de análisis y diagnóstico, habilidades esenciales en la práctica profesional. El producto final constituye una base sólida y extensible, sobre la cual podrían incorporarse mejoras futuras como la implementación de seguridad de nivel productivo mediante tokens JWT, la migración a una base de datos persistente como PostgreSQL, o la incorporación de funcionalidades adicionales de búsqueda y ordenamiento.

8. Bibliografía/Webgrafía

A continuación se detallan las fuentes y recursos de documentación oficial consultados durante el desarrollo del proyecto.

- Documentación oficial de Spring Boot. *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/>
- Documentación oficial de Spring Data JPA. <https://docs.spring.io/spring-data/jpa/reference/>
- Documentación oficial de Hibernate ORM. <https://hibernate.org/orm/documentation/>
- Documentación oficial de Java (Oracle). <https://docs.oracle.com/en/java/>
- Documentación oficial de Vite. <https://vite.dev/guide/>
- Documentación oficial de TypeScript. <https://www.typescriptlang.org/docs/>
- Documentación oficial de Tailwind CSS. <https://tailwindcss.com/docs>
- Documentación de Spring Security Crypto (BCrypt). <https://docs.spring.io/spring-security/reference/>
- MDN Web Docs (Mozilla) – Referencia de Fetch API y JavaScript. <https://developer.mozilla.org/>

